Halo2

Prepared by Kirill Sizov

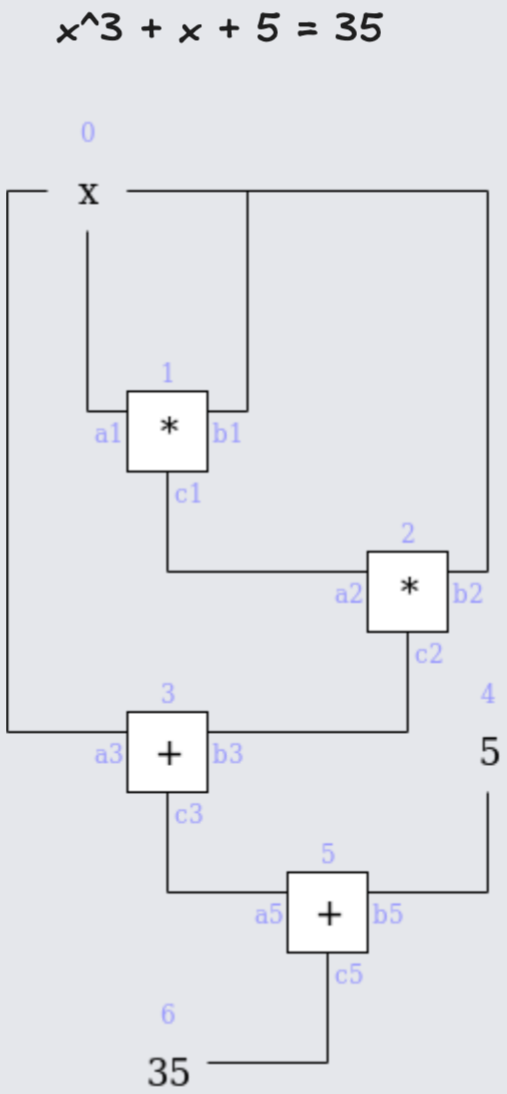
Agenda today

- General about Plonk
- Halo2

Plonk circuits

There are two types of constraints:

1. gate constraints (equations between wires attached to the same gate, e.g. $a_1 \cdot b_1 = c_1$)
2. copy constraints (claims about equality of different wires anywhere in the circuit, e.g. $a_0 = a_1 = b_1 = b_2 = a_3$ or $c_0 = a_1$)



Plonk gates

Each gate can be written in this format:

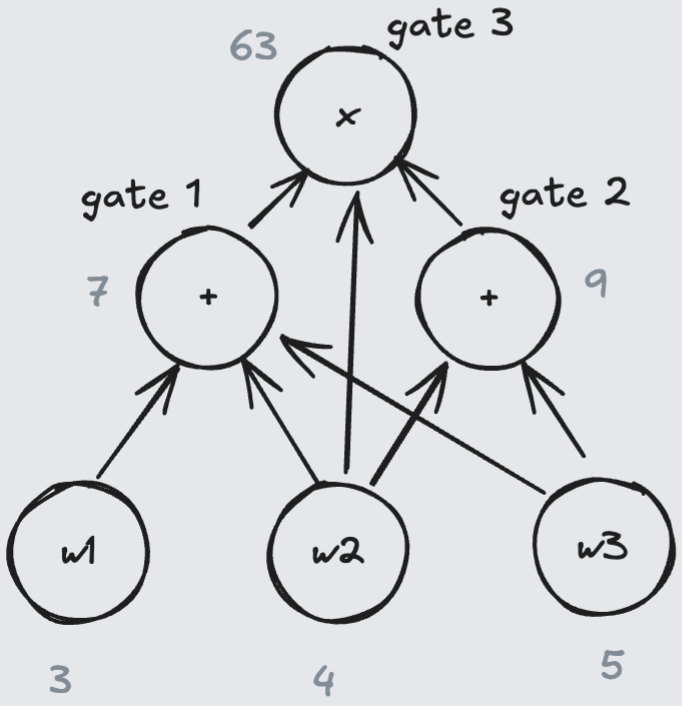
$$(Q_{L_i})a_i + (Q_{R_i})b_i + (Q_{O_i})c_i + (Q_{M_i})a_i b_i + Q_{C_i} = 0$$

Where

- $Q_{L_i}, Q_{R_i}, Q_{M_i}, Q_{O_i}, Q_{C_i}$ – gate selectors
- a_i, b_i, c_i – witness columns

Plonkish aritmetization

(w1 + w2) * (w2 + w3)



Computation trace

	A	B	C	S_add	S_mul
gate 1	3	4	7	1	0
gate 2	4	5	9	1	0
gate 3	7	9	63	0	1

Custom gate

- A custom gate is a user-defined polynomial constraint that must evaluate to 0 on selected rows of the circuit table.
- Allows more specialized constraints, often capturing higher-degree relations in a single gate.

selector column

a_0	a_1	a_2	a_i	s_1
$input_0$	$input_1$	$input_2$		1
va_1	vb_1	vc_1		0
va_2	vb_2	vc_2		0
va_3	vb_3	vc_3		1
va_4	vb_4	vc_4	.....	0
va_5	vb_5	vc_5		0
va_6	vb_6	vc_6		1
va_7	vb_7	vc_7		0

$va6 * vb6 * vc6 - vb7 = 0$

Permutation

- The permutation argument is the mechanism that proves certain cells in the circuit are constrained to carry the same value.

a_0	a_1	a_2	a_i	a_n
$input_0$	$input_1$	$input_2$		output
va_1	vb_1	vc_1		vd_1
va_2	vb_2	vc_2		vd_2
va_3	vb_3	vc_3		vd_3
va_4	vb_4	vc_4	.....	vd_4
va_5	vb_5	vc_5		vd_5
va_6	vb_6	vc_6		vd_6
va_7	vb_7	vc_7		vd_7

$vb2 = vd3 = vc7 = va5$

Lookup

- The lookup argument is the mechanism that proves a value (or tuple of values) used in the circuit appears in a predefined table.

a_0	a_1	a_2	a_i	a_n	T_0
$input_0$	$input_1$	$input_2$		output	000
va_1	vb_1	vc_1		vd_1	001
va_2	vb_2	vc_2		vd_2	010
va_3	vb_3	vc_3	lookup	vd_3	011
va_4	vb_4	vc_4	.....	vd_4	100
va_5	vb_5	vc_5		vd_5	101
va_6	vb_6	vc_6		vd_6	110
va_7	vb_7	vc_7		vd_7	111

vc3 in [0, 8)

Lookup

- The lookup argument is the mechanism that proves a value (or tuple of values) used in the circuit appears in a predefined table.

a_0	a_1	a_2	a_3	a_4	T_0	T_1	T_2
$input_0$	$input_1$	$input_2$		output	000	000	000
va_1	vb_1	vc_1		vd_1	000	001	001
va_2	vb_2	vc_2		vd_2	000	010	010
va_3	vb_3	vc_3		vd_3	000	011	011
va_4	vb_4	vc_4	.....	vd_4	.....	.....	.....
va_5	vb_5	vc_5		vd_5	111	101	010
va_6	vb_6	vc_6		vd_6	111	110	001
va_7	vb_7	vc_7		vd_7	111	111	000

lookup

va3 xor vb3 = vc3

Fibonacci

- Prove the sequence is Fibonacci
- We use only:
  - sum gates
  - permutation argument to enforce copy constraints between consecutive rows

a_0	a_1	a_2
1	1	2
1	2	3
2	3	5
3	5	8
...	...	...
21	34	55

High-level idea of PLONK

- choose an evaluation domain $H = 1, \omega, \omega^2, \ldots, \omega^{n-1}$, where ω is an n-th root of unity ($\omega^n = 1$)
- for each row, the gate constraint is $Q_{L,i}a_i + Q_{R,i}b_i + Q_{O,i}c_i + Q_{M,i}a_ib_i + Q_{C,i} = 0$
- interpolate these row values into polynomials:
  - $(a(x), b(x), c(x))$ interpolate the witness columns,
  - $(Q_L(x), Q_R(x), Q_O(x), Q_M(x), Q_C(x))$ interpolate the selector columns.
  - at each domain point ω^i
    - $a(\omega^i) = a_i, \quad b(\omega^i) = b_i, \quad c(\omega^i) = c_i$
  - same for the selector polynomials.
- the gate constraints can be written as $P_{\text{gate}}(x) = Z_H(x)t(x)$, where $Z_H(x) = x^n - 1$

Halo2

Column types

Type	Defined by	Purpose
Advice	prover (private)	Witness values, the private data the prover knows.
Instance	verifier (public)	Public inputs and outputs. Both prover and verifier know these.
Fixed	circuit setup	Constants for the circuit. Lookup tables are also stored in fixed columns.
Selector	circuit setup	Flag that turns gates on or off for a given row.

Chips

A Chip is a design pattern for packaging a reusable piece of circuit logic. A chip typically has:

- A Config struct, which declares columns and selectors it uses.
- A `configure()` function, which sets up gates and returns the config.
- Assignment functions, which fill in the witness values for specific regions.

Region and layouter

- Region is a rectangular block of cells in the table. Inside a region, you address cells with local offsets starting from 0.
- The Layouter is the algorithm that takes all your regions and maps them to physical rows in the global table.
- `SimpleFloorPlanner` layouter uses a simple strategy: it stacks regions one after another, top to bottom.

Layouter has two key methods:

- `layouter.assign_region()` — assign region to the table.
- `layouter.constrain_instance(...)` — link this advice cell to a public input row.

Circuit trait

The `Circuit` trait has three required methods:

- `configure()` — called once to set up the constraint system (columns, gates). No witness values here.
- `synthesize()` – called to fill in the witness. This is where the prover "runs" the computation.
- `without_witnesses()` — returns a version of the circuit with no witness data, used for key generation.

Steps to implement a circuit

1. Define a `Config` struct that lists all columns (advice, instance, fixed), selectors, and lookup tables your circuit needs. If you use multiple chips, each chip has its own config, and the top-level config composes them.
2. Define one or more chips, each with a `configure()` method that sets up its gates and lookups, and assignment methods that fill in witness values and create copy constraints.
3. Implement the `Circuit` trait: `configure()` sets up all chips, `synthesize()` constructs them, assigns regions through the layouter, and exposes public outputs.
4. Instantiate the circuit with witness data, provide public inputs, and feed both into the proving system.

Example 1

Custom gate:

$$s(a + b - c) = 0$$

Row	col_a	col_b	col_c	selector	instance
0	**1**	**1**	2	1	1 (F[0])
1	1	2	3	1	1 (F[1])
2	2	3	5	1	55 (F[9])
3	3	5	8	1	
4	5	8	13	1	
5	8	13	21	1	
6	13	21	34	1	
7	21	34	**55**	1	
8-15	0	0	0	0	

Example 2

Custom gate:

$$s_i(a_i + a_{i+1} - a_{i+2}) = 0$$

Row	advice	selector	instance
0	**1**	1	1 (F[0])
1	**1**	1	1 (F[1])
2	2	1	55 (F[9])
3	3	1	
4	5	1	
5	8	1	
6	13	1	
7	21	1	
8	34	0	
9	**55**	0	
10-15	0	0	